ADrive Linear Actuator Quick Start (SDK 0.1)

1. What you need

Item	Details
PC	Windows 10/11 64-bit, Python 3.10 or newer (one wheel covers 3.10–3.13)
Actuator	ADrive-LH42 + 48 V DC supply
Link	USB cable (default) or a CAN adapter — see section 4
Safety	**Power-cut emergency stop switch** (mandatory). The SDK's `estop()` is a software stop, not safety rated
Mounting	Clamp the actuator body (vise, clamps, bolts). Never mount with double-sided tape
Firmware	SDK connection confirmed on firmware build 06 and later. Earlier production firmware also works. Some test builds just before 06 do not connect (see `E111` in section 9)

2. Install

```
pip install adrive-0.1.1-cp310-abi3-win_amd64.whl
python -c "import adrive; print(adrive.build_info())"
```

- **No driver installation.** The actuator provides an MS OS descriptor (compatible ID `WINUSB`), so Windows 8.1+ binds WinUSB automatically (no Zadig).
- A COM port may also appear in Device Manager. The SDK does not use it.
- Quote `build_sha` from `build_info()` when asking for support.

3. First connection — home first

Before powering up, place the carriage near the middle of the stroke, clear of both ends. At power-up the actuator moves briefly on its own to find the encoder index and align the encoder (normal; see section 8, item 7). After that, **homing must run once before any motion command is accepted.** Homing moves slowly (2 mm/s) to the endstop switch at the retracted end and defines that point as 0 mm.

```
import adrive

print(adrive.scan())                # ['usb:<serial>'] — attached actuators
with adrive.connect() as act:       # picks the only one; with several use connect("usb:<serial>")
    print(act.info())               # model, serial, firmware
    act.home()                      # once after power-up (up to 60 s)
    act.enable()                    # start control, holding the current position
    act.move_to(10.0)               # go to 10 mm (default 20 mm/s, 100 mm/s^2)
    print(act.position())           # mm
    act.move_to(3.0, speed=50, accel=500)
# leaving the with-block switches the motor off (disable)
```

- Units are always **mm, mm/s, mm/s², N, s**.
- Travel (soft limits) defaults to 0.5–65 mm. `act.set_soft_limits(2, 20)` can narrow it, never widen it.
- Speed/acceleration caps are 200 mm/s and 2000 mm/s². When omitted, 10 % of the caps is used.
- `move_to(..., wait=True)` (default) returns once within ±10 µm of the target, or once the trajectory has ended and the axis has been still for 0.5 s. Friction can leave a few to a few tens of µm; check with `position()`.

4. USB / CAN detection

> **CAN axes cannot move yet (current firmware).** Homing over CAN is not supported by the current firmware, so the SDK refuses it. The SDK also refuses motion on an axis it has not homed. CAN axes can therefore be discovered, monitored and stopped, but not moved. **Use USB for motion** until a later firmware release enables CAN motion. Real CAN adapters are untested on hardware in this release.

Link	How	Status
USB	`adrive.connect()` / `connect("usb:<serial>")` — automatic	Supported
CAN	auto-detect (heartbeat) or `connect("can:<backend>:<channel>:<node id>")`	Discover, monitor, stop only. No motion (see above). Untested on real adapters

Supported CAN adapters

Adapter	Backend	Example target	Notes
PEAK PCAN-USB	`pcan`	`can:pcan:usb1:16`	Install PEAK's PCAN-Basic driver (DLL) first
candleLight / gs_usb (e.g. CANable with candleLight firmware)	`gsusb`	`can:gsusb:0:16`	Needs the WinUSB driver
slcan (CANable stock firmware)	`slcan`	`can:slcan:COM5:16`	Appears as a COM port
Linux SocketCAN	`socketcan`	`can:socketcan:can0:16`	Bring the interface up at 1 Mbit/s beforehand

- Bitrate is **1 Mbit/s**.
- Auto-detect listens for the drive heartbeat for about 1.6 s. Axes are identified by CAN node id (default 16 = 0x10).
- Serial number and firmware version are not available over CAN; only the node id is.
- A node id that looks shared by two drives is reported and left out of auto-detect (an explicit `can:...` target still connects it). Give every unit on a bus its own node id.
- Set the environment variable `ADRIVE_CAN_SCAN=0` to skip CAN probing.
- Several units: `adrive.connect_all()` opens every USB actuator and every CAN axis found by auto-detect (CAN is skipped with `ADRIVE_CAN_SCAN=0`). Add CAN nodes it did not find with `connect_all(["can:pcan:usb1:16", "can:pcan:usb1:17"])`.

5. Force control

```
act.force_control(10.0, max_speed=2.0)      # push with +10 N, at most 2 mm/s
```

- **Command lease**: a force command is valid for 1 s by default. Call `force_control` again within that time to keep pushing. If your script stalls or crashes, the actuator holds its position.

```
end = time.time() + 5
while time.time() < end:
    act.force_control(10.0, max_speed=2.0, lease=0.5)
    print(act.force())                      # N (estimate)
    time.sleep(0.1)
act.stop()                                  # hold the current position
```

- `force()` is an **estimate** from motor current. Screw friction is not compensated; errors of tens of N are possible. Precise force (±1 N) needs a load cell. Force accuracy has not yet been verified with a load cell.
- In force mode, crossing a soft limit stops there and switches to position hold. Pushing against nothing (no load) moves up to the speed cap and stops at the limit.

5-1. Stroke limits

```
act.set_stroke_limits(2.0, 20.0)              # mm from home, 0 <= min < max <= 65
act.set_stroke_limits(2.0, 20.0, save=True)   # remembered on this PC for this unit, applied at next connect
print(act.get_stroke_limits())
```

- A `move_to` outside the limits is **rejected** (`SafetyError`, no motion), not clamped.
- In force mode, crossing a limit switches to holding position there.
- The drive has no position-limit setting; the SDK enforces it. `save=True` stores on **this PC** (`%APPDATA%\adrive\stroke-limits.txt`), not in the drive. Other PCs and other tools do not see it.

5-2. Gain tuning

```
print(act.get_gains())        # {'pos': 300.0, 'vel': 0.015, 'vel_integrator': 0.02}
print(act.gain_limits())      # allowed range
act.set_gains(vel=0.02)       # RAM only, lost at power-off
act.restore_factory_gains()   # back to the shipped values (RAM)
act.disable(); act.save()     # store in the drive — the drive reboots
```

- Allowed range: **0.2–5x the factory value**, within the absolute bounds (pos 20–1500, vel 0.0005–0.08, vel_integrator 0.001–0.1). Values outside are rejected.
- The SDK checks the range on every firmware. From firmware build 06 the drive also rejects out-of-range gain writes; on earlier firmware the drive does not check writes that bypass the SDK.
- Factory gains are the per-unit certificate values and differ from unit to unit (the example above is one unit). Use `restore_factory_gains()` to go back to them.
- The factory value is what the drive reports at connect. After a `save()`, set the certificate values with `act.set_factory_gains(pos, vel, vel_integrator)`.
- `save()` works only while disabled. The drive reboots and **may move during its startup sequence (encoder index search)**. Clear the stroke, then `connect()` again, check `get_gains()`, home.
- Change in small steps (within 1.5x) and check the step response (overshoot, settling time) in the GUI.

6. Four-axis synchronized example

```
python examples/sync_4axis.py                    # every USB axis
python examples/sync_4axis.py --extra can:pcan:usb1:16   # add CAN nodes (no motion yet — section 4)
python examples/sync_4axis.py --sim 3            # add simulated axes (no hardware)
```

Homes all axes at once, then moves each axis a different distance in the same time. Per move it prints the start sync error (spread of command and actual start times), axis-to-axis phase error (spread of midpoint crossing times) and the worst deviation from the ideal profile. Any axis error stops every axis immediately.

With the current firmware, a CAN axis in the mix has its homing refused, so the example stops every axis and exits (section 4). For now, move USB axes only. CAN wiring and per-adapter notes are in `examples/sync_4axis.py --help`.

7. GUI

A single Windows executable, `ADrive-GUI.exe`, runs without installation. It is built with Tk and uses the public SDK only.

Screen / function	Details
Device list	USB and CAN auto-detect. Shows serial, model, firmware and link
Connect / disconnect	Pick a device from the list

Screen / function	Details
Homing	Home button and homing badge. Motion buttons stay disabled until homed
Enable / Disable	Switch motor control on and off
Move	Target in mm, presets (0–100 % of the soft-limit range), jog ±0.01 / 0.1 / 1 / 10 mm
Speed / acceleration	Sliders, capped by the SDK limits
Force tab	**EXPERIMENTAL** (until the load-cell test). Uses the command lease
Live plot	Position, target, current, estimated force
Fault display	Shows drive faults
EMERGENCY STOP	Big button, keys Space / Esc. Sets the motor idle on all axes
CSV logging	Saves measurements to CSV
Multi-axis	Up to 4 axes, with synchronized moves

- The GUI EMERGENCY STOP is a **software stop**, not a safety-rated emergency stop. Keep the power-cut emergency stop switch.
- The CAN limitation in section 4 applies to the GUI as well.
- Only one program can open an actuator at a time. Do not open the same unit from a Python script while the GUI has it.
- From source, with 4 simulated axes and no hardware: `python gui/adrive_gui.py --sim 4`

8. Safety rules

1. For the first moves use low speed (≤ 10 mm/s) and short distances (1–5 mm) and check the direction. Positive = away from the endstop (extend).
2. Keep hands and body out of the rod's travel. Keep the power-cut emergency stop within reach.
3. **Thermal protection**: in this version the drive temperature reading is invalid, so there is no over-temperature protection. Do not run long continuous high-speed cycles; run briefly and pause.
4. Use soft limits to stay clear of the stroke ends (endstop at 0 mm, mechanical end near 69 mm).
5. If your program dies, the motor switches off within 0.5 s (communication watchdog). A switched-off motor produces no force: in vertical mounting or under external load the rod can be pushed.
6. No firmware or configuration tools are provided. Do not open or modify the actuator.
7. **Power-up encoder search**: right after power is applied the drive searches for the encoder index and calibrates by itself, and the carriage moves briefly. Before power-up, keep the carriage at least 15 mm from either end and keep it free of the end stop, vise, fixture or load. If it is blocked, the travel is too short and the drive reports `ENCODER_FAILED` (0x100) with encoder error `CPR_POLEPAIRS_MISMATCH` (0x2) and stays disabled (see section 9).

9. Troubleshooting

Symptom	Cause / action
`NotFoundError`	Check USB cable and power. If `adrive.scan()` is empty, check Device Manager
`LinkError: E102 device busy`	Another program (another Python, the GUI) already has the actuator open. One program at a time
`SafetyError: not homed`	Run `home()` first after power-up
`SafetyError: ... outside soft limits` / speed	Target or speed outside the allowed range shown in the message
`FaultError (code ...)`	The actuator stopped itself. `disable()` → `clear_faults()` → check the cause → `enable()`.

Symptom	Cause / action
	Quote the code to support
`FaultError` on the first connection after a crash	Record of the watchdog stop. Clear it with `clear_faults()`
`TimeoutError` (move_to)	It may be creeping onto the target against friction. Increase `timeout=` or use `wait=False` and check `position()`
`LinkError ... reconnect required`	The link dropped. Check the cable and call `connect()` again (no automatic reconnect — by design)
`LinkError: E111` at connect	The drive runs a test firmware build that does not support the SDK. Ask support for firmware build 06 or later
Right after power-up: encoder error `CPR_POLEPAIRS_MISMATCH` (0x2) / `ENCODER_FAILED` (0x100), motor stays off	The carriage was blocked during the power-up encoder search (resting on the end stop, vise or fixture contact, connected load), so the measured travel was too short. Cut power, move the carriage by hand at least 15 mm from the end stop, remove the obstruction, and power up again. If it recurs, check the encoder connector. This is not a stored-settings problem

10. License notices

- The SDK, GUI, documents and actuator firmware are AeiROBOT proprietary software, licensed under a separate agreement (EULA).
- Notices for third-party libraries in the SDK (Rust crates: pyo3, nusb, serde_json, …; MIT/Apache-2.0) ship as `THIRD_PARTY_NOTICES.txt`.